Package 'lpmec'

July 11, 2026

Title Measurement Error Analysis and Correction Under Identification Restrictions

Version 1.3.0

Author Connor Jerzak [aut, cre], Stephen Jessee [aut]

Description Implements methods for analyzing latent variable models with measurement error correction, including Item Response Theory (IRT) models. Provides tools for various correction methods such as Bayesian Markov Chain Monte Carlo (MCMC), over-imputation, bootstrapping for robust standard errors, Ordinary Least Squares (OLS), and Instrumental Variables (IV) based approaches. Supports flexible specification of observable indicators and groupings for latent variable analyses in social sciences and other fields. Methods are described in a working paper (2025) <doi:10.48550/arXiv.2507.22218>. Also includes corrections for panel fixed-effects and differencing designs, latent treatment-effect moderators in randomized experiments, and multi-measure reliability bounds.

Depends R (>= 3.5.0)

License GPL-3

Encoding UTF-8

LazyData false

Maintainer Connor Jerzak <connor.jerzak@gmail.com>

Imports reticulate,
stats,
sensemakr,
pscl,
AER,
sandwich,
mvtnorm,
Amelia,
emIRT,
gtools

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown

SystemRequirements Python (>= 3.10) with jax, numpy, numpyro (optional; for NumPyro backend via reticulate)

VignetteBuilder knitr

Config/testthat/edition 3

URL https://github.com/cjerzak/lpmec-software

BugReports https://github.com/cjerzak/lpmec-software/issues
Config/roxygen2/version 8.0.0

Contents

`build_backend` *A function to build the environment for lpmec. Builds a conda environment in which 'JAX', 'numpyro', and 'numpy' are installed. Users can also create a conda environment where 'JAX' and 'numpy' are installed themselves.*

Description

A function to build the environment for lpmec. Builds a conda environment in which 'JAX', 'numpyro', and 'numpy' are installed. Users can also create a conda environment where 'JAX' and 'numpy' are installed themselves.

Usage

```
build_backend(conda_env = "lpmec", conda = "auto")
```

Arguments

conda_env	(default = "lpmec") Name of the conda environment in which to place the backends.
conda	(default = auto) The path to a conda executable. Using "auto" allows reticulate to attempt to automatically find an appropriate conda binary.

Value

Invisibly returns NULL; this function is used for its side effects of creating and configuring a conda environment for lpmec. This function requires an Internet connection. You can find out a list of conda Python paths via: Sys.which("python")

Examples

```
## Not run:
# Create a conda environment named "lpmec"
# and install the required Python packages (jax, numpy, etc.)
build_backend(conda_env = "lpmec", conda = "auto")

# If you want to specify a particular conda path:
# build_backend(conda_env = "lpmec", conda = "/usr/local/bin/conda")

## End(Not run)
```

---

infer_orientation_signs

Infer orientation signs for each observable indicator

Description

This helper analyzes observable indicators and returns a numeric vector of 1 or -1 for use with the orientation_signs argument in lpmec. Each sign is chosen so that the correlation between the oriented indicator and either the outcome Y or the first principal component of the indicators is positive.

Usage

```
infer_orientation_signs(Y, observables, method = c("Y", "PC1"))
```

Arguments

Y	Numeric outcome vector. Only used when method = "Y".
observables	A matrix or data frame of binary observable indicators.
method	Character string specifying how to orient the indicators.

`"Y"` orient each indicator so that its correlation with `Y` is positive. Caution: this uses the outcome in the construction of the measurement, which is in mild tension with the assumption that measurement error is independent of the outcome error; prefer `"PC1"` when the indicators alone determine the orientation.

`"PC1"` orient each indicator so that its correlation with the first principal component of `observables` is positive.

Default is `"Y"`.

Value

A numeric vector of length `ncol(observables)` containing `1` or `-1`.

Examples

```
set.seed(1)
Y <- rnorm(10)
obs <- data.frame(matrix(sample(c(0,1), 20, replace = TRUE), ncol = 2))
infer_orientation_signs(Y, obs)
```

`KnowledgeVoteDuty` *KnowledgeVoteDuty: Survey Respondents' Views of Voting as a Duty and Political Knowledge Questions*

Description

KnowledgeVoteDuty is a modified set of responses to a small set of questions on the American National Election Study's 2024 Time Series Study. These data only include respondents who had non-missing values on all of the variables included, dropping respondents with one or more missing values.

Usage

```
data(KnowledgeVoteDuty)
```

Format

A data frame with 3,059 observations and 5 variables:

voteduty Whether respondents feel that voting is a duty or a choice. Values range from 1 to 7, with 1 being "Very strongly a duty" and 7 being "Very strongly a choice," created based on variable V241218x.

SenateTerm Dummy variable (0 or 1) for whether respondent correctly stated the length of a U.S. Senate term. Created based on variable V241612.

SpendLeast Dummy variable (0 or 1) for whether respondent correctly identified "Foreign aid" from a list as the category the federal government spends the least on. Created based on variable V241613.

HouseParty Dummy variable (0 or 1) for whether respondent correctly identified the political party that currently has the most members in the U.S. House of Representatives. Created based on variable V241614.

SenateParty Dummy variable (0 or 1) for whether respondent correctly identified the political party that currently has the most members in the U.S. Senate. Created based on variable V241615.

References

American National Election Studies. 2024. ANES 2024 Time Series Study Full Release [dataset and documentation]. Available at electionstudies.org.

Examples

```
data(KnowledgeVoteDuty)
voteduty <- KnowledgeVoteDuty$voteduty
knowledge <- scale(rowMeans(KnowledgeVoteDuty[ , -1]))
summary(lm(voteduty ~ knowledge))
```

`lpmec` *lpmec*

Description

Implements latent variable models with measurement error correction

Usage

```
lpmec(
  Y,
  observables,
  observables_groupings = colnames(observables),
  orientation_signs = NULL,
  make_observables_groupings = FALSE,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  boot_basis = 1:length(Y),
  bootstrap_method = c("n_out_of_n", "m_out_of_n", "subsampling", "auto"),
  boot_m = NULL,
  boot_m_rule = c("power", "fixed", "grid_stability"),
  boot_m_exponent = 0.7,
  boot_m_grid = NULL,
  boot_m_replace = NULL,
  boot_ci_type = c("auto", "root", "root_calibrated", "percentile", "rbc"),
  boot_alpha = 0.05,
  boot_rate = c("sqrt_n", "custom"),
  boot_tau = NULL,
  boot_calibration_n = 25L,
  boot_calibration_inner_n_boot = NULL,
  boot_calibration_cut_grid = c(0.005, 0.01, 0.015, 0.02, 0.025, 0.035, 0.05),
  boot_calibration_tail = c("separate", "equal_tail"),
  boot_calibration_seed = NULL,
  boot_rbc_item_counts = NULL,
  boot_rbc_n_subsets = 10L,
  boot_rbc_seed = NULL,
  partition_set = NULL,
```

```
  fix_partitions = TRUE,
  seed = NULL,
  return_intermediaries = TRUE,
  ordinal = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

`Y`	A vector of observed outcome variables
`observables`	A matrix of observable indicators used to estimate the latent variable
`observables_groupings`	A vector specifying groupings for the observable indicators. Default is column names of observables.
`orientation_signs`	(optional) A numeric vector of length equal to the number of columns in 'observables', containing 1 or -1 to indicate the desired orientation of each column. If provided, each column of 'observables' will be oriented by this sign before analysis. Default is NULL (no orientation applied).
`make_observables_groupings`	Logical. If TRUE, creates dummy variables for each level of the observable indicators. Default is FALSE.
`n_boot`	Non-negative integer. Number of bootstrap iterations. Use `0` to disable bootstrap resampling and fit only the original sample. Default is 32.
`n_partition`	Positive integer. Number of split-half partitions for each bootstrap iteration. When `n_boot = 0`, this still controls how many original-sample partition runs are aggregated. Default is 10.
`partition_aggregation`	Aggregation strategy for combining estimates across partitions within each bootstrap iteration. Default is `"median"`. Options are `"median"`, `"winsorized_mean"`, `"trimmed_mean"`, or a custom function that accepts a numeric vector and returns one numeric value.
`partition_aggregation_probs`	Numeric vector of length 2 used by `"winsorized_mean"` and `"trimmed_mean"`. For winsorization, values are clipped to these quantiles before averaging. For trimming, values outside these quantiles are dropped before averaging. Default is `c(0.01, 0.99)`.
`boot_basis`	Vector of indices or grouping variable for stratified bootstrap. Default is 1:length(Y).
`bootstrap_method`	Resampling method for uncertainty. Options are `"n_out_of_n"`, `"m_out_of_n"`, `"subsampling"`, and `"auto"`. The default `"n_out_of_n"` preserves historical row bootstrap behavior. Use `"subsampling"` or `"m_out_of_n"` with `boot_ci_type = "root"` for the formal nonsmooth median route.
`boot_m`	Optional exact m for m-out-of-n bootstrap or subsampling.

boot_m_rule Rule used when boot_m = NULL. "power" uses floor(n^boot_m_exponent); "fixed" requires boot_m; "grid_stability" records a candidate grid and currently selects the grid value nearest the power-rule value.

boot_m_exponent
Exponent used by boot_m_rule = "power". Default is 0.70.

boot_m_grid Optional candidate grid for m-sensitivity diagnostics.

boot_m_replace Optional logical override for row replacement in m-out-of-n/subsampling. By default, replacement is used for "n_out_of_n" and "m_out_of_n", and not used for "subsampling".

boot_ci_type Confidence interval type. "auto" uses percentile intervals for "n_out_of_n" and root-scaled intervals for m < n. "root_calibrated" uses nested subsampling to calibrate the root interval tail cutoffs. "rbc" uses finite-item-count robust bias correction with root-scaled intervals.

boot_alpha Confidence interval tail probability. Default is 0.05.

boot_rate Rate used by root-scaled intervals. The current formal path uses "sqrt_n"; "custom" requires boot_tau.

boot_tau Optional function used when boot_rate = "custom".

boot_calibration_n
Positive integer. Number of outer calibration subsamples used when boot_ci_type = "root_calibrated". Default is 25.

boot_calibration_inner_n_boot
Optional positive integer. Number of inner resamples within each calibration subsample. Defaults to n_boot.

boot_calibration_cut_grid
Candidate one-sided tail probabilities for nested root interval calibration. Values must be greater than 0 and less than or equal to boot_alpha. Default is c(0.005, 0.010, 0.015, 0.020, 0.025, 0.035, 0.050).

boot_calibration_tail
Calibration mode. "separate" calibrates lower and upper one-sided root intervals separately; "equal_tail" chooses one common equal-tail cutoff.

boot_calibration_seed
Optional seed for the nested calibration stage.

boot_rbc_item_counts
Optional integer vector of observable-group counts used to estimate the leading finite-M bias when boot_ci_type = "rbc". Defaults to a fixed grid based on fractions of the full number of observable groups.

boot_rbc_n_subsets
Positive integer. Number of item subsets drawn at each value of boot_rbc_item_counts. Used only when boot_ci_type = "rbc". Default is 10.

boot_rbc_seed Optional seed for the item-subset schedule used by boot_ci_type = "rbc".

partition_set Optional user-supplied fixed partition list. Each element must contain split1_names, split2_names, and optionally partition_id.

fix_partitions Logical. If TRUE, draw or accept the finite partition set once and hold it fixed across original and resampled fits.

seed Optional seed used for reproducible partitions and resamples.

return_intermediaries
Logical. If TRUE, returns intermediate results. Default is TRUE.

ordinal Logical indicating whether the observable indicators are ordinal (TRUE) or binary (FALSE).

estimation_method
Character specifying the estimation approach. Options include:

- "em" (default): Uses expectation-maximization via emIRT package. Supports both binary (via emIRT::binIRT) and ordinal (via emIRT::ordIRT) indicators.
- "pca": First principal component of observables.
- "averaging": Uses feature averaging.
- "mcmc": Markov Chain Monte Carlo estimation using either pscl::ideal (R backend) or numpyro (Python backend)
- "mcmc_joint": Joint Bayesian model that simultaneously estimates latent variables and outcome relationship using numpyro
- "mcmc_joint2": NumPyro mixed factor-analysis benchmark with binary indicators and continuous Y in one factor model
- "mcmc_overimputation": Two-stage MCMC approach with measurement error correction via over-imputation
- "custom": In this case, latent estimation performed using latent_estimation_fn.

latent_estimation_fn
Custom function for estimating latent trait from observables if estimation_method="custom" (optional). The function should accept a matrix of observables (rows are observations) and return a numeric vector of length equal to the number of observations.

mcmc_control A list indicating parameter specifications if MCMC used.

backend Character string indicating the MCMC engine to use. Valid options are "pscl" (default, uses the R-based pscl::ideal function) or "numpyro" (uses the Python numpyro package via reticulate).

n_samples_warmup Integer specifying the number of warm-up (burn-in) iterations before samples are collected. Default is 500.

n_samples_mcmc Integer specifying the number of post-warmup MCMC iterations to retain. Default is 1000.

batch_size Integer row subsample size used when subsample_method = "batch" with the NumPyro backend. Must be between 1 and nrow(observables) - 1. Default is 512.

subsample_method Character string for NumPyro likelihood evaluation. Use "full" (default) for all rows or "batch" for experimental HMCECS row subsampling with batch_size.

chain_method Character string passed to numpyro specifying how to run multiple chains. Options: "parallel" (default), "sequential", or "vectorized".

anchor_parameter_id Optional 1-based observable-column index used by NumPyro MCMC backends to anchor item difficulty orientation. If omitted or NULL, automatic orientation is used.

n_thin_by Integer indicating the thinning factor for MCMC samples. Default is 1.

n_chains Integer specifying the number of parallel MCMC chains to run. Default is 2.

outcome_prior List controlling "mcmc_joint" outcome-model priors for the NumPyro backend. By default, calibration = "data" centers the intercept prior at mean(Y) and scales intercept, slope, and residual-sigma priors

by sd(Y). Use calibration = "legacy" to restore the previous unit-scale priors, or provide numeric overrides for intercept_mean, intercept_sd, slope_mean, slope_sd, and sigma_sd. The optional scale_floor sets the minimum scale used for data-calibrated priors.

joint2_prior List controlling "mcmc_joint2" priors. Defaults are lambda_mean = 0, lambda_sd = 2, psi_shape = 0.0005, and psi_scale = 0.0005. For item loadings, lambda_mean and lambda_sd parameterize the raw loading scale before the positive softplus transform; all item loadings are positive by default.

conda_env A character string specifying the name of the conda environment to use via reticulate. Default is "lpmec".

conda_env_required
A logical indicating whether the specified conda environment must be strictly used. If TRUE, an error is thrown if the environment is not found. Default is FALSE.

Details

This function implements a latent variable analysis with measurement error correction. It fits the original sample and, when n_boot >= 1, performs bootstrap resampling for uncertainty estimates. Each original or bootstrap sample is analyzed with one or more split-half partitions. For each partition, it calls the lpmec_onerun function to estimate latent variables and apply various correction methods. The results are then aggregated across partitions and bootstrap iterations to produce final estimates and, when bootstrap draws are available, bootstrap standard errors.

For partition_aggregation = "median", the finite-partition median is a nonsmooth aggregation rule. The ordinary n-out-of-n row bootstrap remains available through bootstrap_method = "n_out_of_n" for backward compatibility. The formal nonsmooth-functional route is bootstrap_method = "subsampling" or "m_out_of_n" with boot_ci_type = "root" or "root_calibrated". In that route, the same realized partition set is held fixed across the original sample and all resamples, each resample reruns the full latent-score and correction pipeline, and confidence intervals invert the empirical distribution of sqrt(m) * (theta_boot - theta_hat) at the original sqrt(n) rate.

With boot_ci_type = "root_calibrated", lpmec() performs a nested subsampling calibration of the root interval cutoffs. For each calibration subsample of size m, it treats the full-sample estimate as a pseudo-truth, builds inner root intervals from smaller subsamples, estimates candidate one-sided coverage rates, and applies the largest candidate cutoff whose estimated coverage reaches the nominal target. This is an asymptotic resampling calibration, not a finite-sample coverage guarantee. Its usual justification relies on scale separation for the outer and inner subsample sizes.

With boot_ci_type = "rbc", lpmec() estimates a leading finite-item-count bias by rerunning the estimator on a fixed, pre-specified grid of observable-group subsets and fitting each target summary to a + b / M. It subtracts b / M from the full-M estimate and constructs root intervals from bootstrap draws of the same bias-corrected statistic.

Value

A list containing various estimates and statistics (in snake_case):

- Naive, IV, corrected IV, and corrected OLS estimates: ols_*, iv_*, corrected_iv_*, and corrected_ols_*. Bootstrap uncertainty summaries use suffixes _se, _lower, _upper, and _tstat where applicable.
- var_est_split and var_est_split_se: Aggregated split-half measurement-error variance and, when bootstrap draws are available, its bootstrap standard error.

- bayesian_ols_*_outer_normed and bayesian_ols_*_inner_normed: MCMC coefficient summaries. The *_parametric standard-error fields retain within-run posterior uncertainty, while the non-parametric standard-error and interval fields summarize bootstrap variation.
- m_stage_1_erv* and m_reduced_erv*: Extreme robustness values and bootstrap uncertainty summaries for the first-stage and reduced-form regressions.
- mcmc_joint2_*: NumPyro "mcmc_joint2" diagnostics, including effective-sample-size percentages, maximum R-hat, divergent transitions, mean accept probability, and orientation diagnostics.
- x_est1 and x_est2: Split-half latent variable estimates from the original sample.
- boot_rbc_diagnostics, boot_rbc_raw_aggregates, and boot_rbc_pilot_aggregates: Robust-bias-correction diagnostics returned when boot_ci_type = "rbc".
- Intermediary_*: Per-run original-sample and bootstrap outputs, returned only when return_intermediaries = TRUE.

References

Jerzak, C. T. and Jessee, S. A. (2025). Attenuation Bias with Latent Predictors. arXiv:2507.22218 [stat.AP]. https://arxiv.org/abs/2507.22218

Calonico, S., Cattaneo, M. D., and Farrell, M. H. (2018). On the Effect of Bias Estimation on Coverage Accuracy in Nonparametric Inference. Journal of the American Statistical Association, 113, 767–779.

Examples

```
# Generate some example data
set.seed(123)
Y <- rnorm(1000)
observables <- as.data.frame(matrix(sample(c(0,1), 1000*10, replace = TRUE), ncol = 10))

# Run the bootstrapped analysis
results <- lpmec(Y = Y,
                 observables = observables,
                 n_boot = 10,    # small values for illustration only
                 n_partition = 5 # small for size
                 )

# Use a winsorized mean across partitions
results_winsorized <- lpmec(Y = Y,
                            observables = observables,
                            n_boot = 10,
                            n_partition = 5,
                            partition_aggregation = "winsorized_mean")

# View the corrected IV coefficient and its standard error
print(results)
```

`lpmec_moderator` *Aggregated latent-moderator interaction correction with bootstrap*

Description

Runs `lpmec_moderator_onerun` over repeated split-half partitions (in `observables` mode) and row (or stratified) bootstrap samples, re-running the whole pipeline – scoring with fresh splits, reliability estimation, interaction regression, and correction – on every (bootstrap, partition) draw so the reported uncertainty reflects all estimation steps.

Usage

```
lpmec_moderator(
  Y,
  treatment,
  observables = NULL,
  scores = NULL,
  split_scores = NULL,
  covariates = NULL,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  boot_basis = seq_along(Y),
  return_intermediaries = TRUE,
  estimation_method = "averaging",
  min_reliability = 0.05,
  seed = NULL,
  ...
)
```

Arguments

`Y`	Numeric outcome vector with only finite values.
`treatment`	Numeric treatment vector of the same length as `Y` with only finite values and positive variance. Treatments with more than 2 unique values are allowed and treated as continuous (a message is emitted).
`observables`	Optional list of item matrices or data frames, one per measure (a single matrix is treated as one measure). Each measure is scored via `lpmec_onerun` with `estimation_method`, and its full-battery and split-half latent scores are harvested. Each item matrix must have at least 4 columns; measures with fewer components must be supplied through `split_scores`.
`scores`	Optional named list (or matrix/data frame with one column per measure) of pre-computed full measure scores. Scores are pooled z-scored; split-based reliabilities are unavailable for these measures.
`split_scores`	Optional named list of n x 2 matrices of half scores, one per measure. Each half is pooled z-scored and the full-measure score is the z-score of the mean of the two z-scored halves.
`covariates`	Optional matrix or data frame of observed covariates included additively in the interaction regression.

`n_boot` Non-negative integer number of n-out-of-n bootstrap replications. Default `32`.

`n_partition` Positive integer number of fresh split-half partitions per original or bootstrap sample. Fresh partitions require item-level inputs, so when `observables` is `NULL` the value is coerced to 1 with a message. Default `10`.

`partition_aggregation`
Aggregation strategy across partitions within each bootstrap sample; see `lpmec`. Default `"median"`.

`partition_aggregation_probs`
Quantile probabilities for winsorized or trimmed partition aggregation.

`boot_basis` Optional vector of length `length(Y)` of strata labels for the bootstrap. With all-unique values (the default, `seq_along(Y)`), rows are resampled with replacement; otherwise rows are resampled with replacement within each stratum (for example, `boot_basis = treatment` preserves the arm sizes).

`return_intermediaries`
Logical. If `TRUE`, returns per-run vectors/matrices of all aggregated quantities.

`estimation_method`
Estimation method(s) passed to `lpmec_onerun` for `observables`-mode measures (single value or one per measure). Default `"averaging"`.

`min_reliability`
Reliability floor in [0, 1). When `rho_score` (or a per-measure triad reliability) is below this value or not finite, the corresponding corrected coefficient is reported as `NA` with a single warning rather than dividing by a tiny or undefined reliability. Default `0.05`.

`seed` Optional seed applied locally to the partitions and the bootstrap (the caller's random-number state is restored on exit).

`...` Additional arguments passed to `lpmec_onerun` for `observables`-mode measures (e.g., `ordinal`, `mcmc_control`).

Details

The first run (bootstrap index 1) is the original sample; its partition-aggregated estimates are the reported point estimates, and the remaining `n_boot` runs feed the bootstrap standard errors and percentile intervals. Because reliability estimation is re-done on every draw, the intervals for `corrected_interaction_coef` propagate the sampling uncertainty in `rho_half` and `rho_score` – both of which are reported with their own bootstrap summaries as headline quantities. Warnings and messages from the pipeline are shown only for the first (original-sample, first-partition) run; corrected coefficients floored to `NA` in bootstrap replications propagate `NA` through the default median aggregation.

Value

A list of class `lpmec_moderator` containing, for each of `treatment_coef`, `main_coef`, `interaction_coef`, `corrected_main_coef`, `corrected_interaction_coef`, `rho_half`, and `rho_score`: the original-sample estimate (aggregated across partitions) plus bootstrap `_se`, `_lower`, and `_upper` summaries (2.5% and 97.5% percentiles) when `n_boot >= 1`. Also included are `coef_all`/`coef_se_all`, `sb_factor`, `per_measure` and `cor_matrix` from the original-sample run, `x_used`/`x_est`/`x_est1`/`x_est2`, `measure_names`, `n_measures`, `measure_source`, `covariate_names`, `n_obs`, `n_boot`, `n_partition`, `min_reliability`, and – when `return_intermediaries = TRUE` – `Intermediary_BootIndex`, `Intermediary_PartitionIndex`, and per-run `Intermediary_*` vectors (plus `Intermediary_coef_all`).

Examples

```
set.seed(101)
n <- 400
X <- rnorm(n)
treatment <- rbinom(n, 1, 0.5)
Y <- 0.2 * treatment + 0.2 * X + 0.3 * treatment * X + rnorm(n)
items <- matrix(rbinom(n * 4L, 1,
                       stats::pnorm(outer(X, c(1.2, 1, 0.8, 1.1)) - 0.2)),
                n, 4L)
colnames(items) <- paste0("item", 1:4)

result <- lpmec_moderator(
  Y = Y,
  treatment = treatment,
  observables = items,
  estimation_method = "averaging",
  n_boot = 4L,
  n_partition = 2L,
  seed = 7
)
c(corrected = result$corrected_interaction_coef,
  lower = result$corrected_interaction_lower,
  upper = result$corrected_interaction_upper)
```

`lpmec_moderator_onerun` *Single-run latent-moderator interaction correction*

Description

Estimates the interaction between a treatment and a latent moderator that is measured with error, and corrects the naive interaction coefficient for the attenuation induced by the moderator score's reliability. With a randomized treatment and a standardized moderator score `x` of reliability `rho`, the naive interaction coefficient from `Y ~ treatment * x` converges to `b_TX * sqrt(rho)`; the corrected estimate divides by `sqrt(rho_score)`, where `rho_score` is the Spearman-Brown as-used reliability of the score actually entered in the regression.

Usage

```
lpmec_moderator_onerun(
  Y,
  treatment,
  observables = NULL,
  scores = NULL,
  split_scores = NULL,
  covariates = NULL,
  estimation_method = "averaging",
  min_reliability = 0.05,
  ...
)
```

Arguments

Y	Numeric outcome vector with only finite values.
treatment	Numeric treatment vector of the same length as Y with only finite values and positive variance. Treatments with more than 2 unique values are allowed and treated as continuous (a message is emitted).
observables	Optional list of item matrices or data frames, one per measure (a single matrix is treated as one measure). Each measure is scored via lpmec_onerun with estimation_method, and its full-battery and split-half latent scores are harvested. Each item matrix must have at least 4 columns; measures with fewer components must be supplied through split_scores.
scores	Optional named list (or matrix/data frame with one column per measure) of pre-computed full measure scores. Scores are pooled z-scored; split-based reliabilities are unavailable for these measures.
split_scores	Optional named list of n x 2 matrices of half scores, one per measure. Each half is pooled z-scored and the full-measure score is the z-score of the mean of the two z-scored halves.
covariates	Optional matrix or data frame of observed covariates included additively in the interaction regression.
estimation_method	Estimation method(s) passed to lpmec_onerun for observables-mode measures (single value or one per measure). Default "averaging".
min_reliability	Reliability floor in [0, 1). When rho_score (or a per-measure triad reliability) is below this value or not finite, the corresponding corrected coefficient is reported as NA with a single warning rather than dividing by a tiny or undefined reliability. Default 0.05.
...	Additional arguments passed to lpmec_onerun for observables-mode measures (e.g., ordinal, mcmc_control).

Details

The correction divides by the square root of the reliability of the score actually used as the regressor (Proposition 6 of the accompanying working paper). With a single item battery, the split-half correlation rho_half estimates the reliability of a half score, so the full-battery score reliability is the Spearman-Brown step-up 2 * rho_half / (1 + rho_half); dividing by sqrt(rho_half) instead would overcorrect. With M >= 2 measures, the regressor is the mean of the M aligned z-scored measure scores and the analogous generalized step-up M * rbar / (1 + (M - 1) * rbar) is applied to the mean pairwise correlation rbar. When M >= 3, the per-measure triad reliabilities rho*_m = r_ml * r_mk / r_lk – consistent when measurement errors are independent across measures – provide an alternative correction reported in per_measure. Covariates enter the regression additively; the correction is exact when the covariates are independent of the latent moderator and approximate otherwise. Reliabilities below min_reliability (or not estimable, as with a single scores-mode measure) yield NA corrected coefficients and a single warning.

Value

A list of class lpmec_moderator_onerun containing:

- treatment_coef/treatment_se, main_coef/main_se, interaction_coef/interaction_se: naive OLS coefficients and classical standard errors from Y ~ treatment + x + treatment:x (plus covariates), where x is the moderator score in x_used.

- coef_all, se_all: all named coefficients (intercept omitted), including covariates.
- rho_half: with one measure, the split-half correlation of that measure's two half scores; with M >= 2 measures, the mean pairwise correlation among the z-scored, sign-aligned measure scores.
- sb_factor, rho_score: the Spearman-Brown step-up factor (2 for half scores of one battery, M for M averaged measures) and the as-used score reliability rho_score = sb_factor * rho_half / (1 + (sb_factor - 1) * rho_half).
- correction_factor: sqrt(rho_score) when usable, otherwise NA.
- corrected_interaction_coef, corrected_main_coef: interaction_coef / sqrt(rho_score) and main_coef / sqrt(rho_score) (NA when rho_score is below min_reliability or not finite).
- reliability_floored: logical, TRUE when the headline correction was floored to NA.
- per_measure: when M >= 3, a data frame with one row per measure holding the naive per-measure interaction coefficient (and classical standard error), the pooled triad reliability rho*_m, and the triad-corrected interaction coefficient; NULL otherwise.
- cor_matrix: pooled cross-measure correlation matrix (NULL when M == 1).
- x_used: the moderator score entered in the regression (the z-scored full-battery score for one measure; the z-scored mean of the aligned z-scored measure scores for M >= 2).
- x_est, x_est1, x_est2: per-measure pooled z-scored, sign-aligned score matrices (halves are NA for scores-mode measures).
- measure_names, n_measures, measure_source, sign_flipped, covariate_names, n_obs, n_obs_used, min_reliability, scalar_runs.

Examples

```
set.seed(100)
n <- 600
X <- rnorm(n)
treatment <- rbinom(n, 1, 0.5)
Y <- 0.2 * treatment + 0.2 * X + 0.3 * treatment * X + rnorm(n)
items <- matrix(rbinom(n * 4L, 1,
                       stats::pnorm(outer(X, c(1.2, 1, 0.8, 1.1)) - 0.2)),
                n, 4L)
colnames(items) <- paste0("item", 1:4)

run <- lpmec_moderator_onerun(
  Y = Y,
  treatment = treatment,
  observables = items,
  estimation_method = "averaging"
)
c(naive = run$interaction_coef,
  rho_score = run$rho_score,
  corrected = run$corrected_interaction_coef)
```

`lpmec_multivariate` *Aggregated multivariate latent-predictor correction*

Description

Runs `lpmec_multivariate_onerun` over repeated split-half partitions and optional row bootstrap samples.

Usage

```
lpmec_multivariate(
  Y,
  observables,
  covariates = NULL,
  observables_groupings = NULL,
  make_observables_groupings = FALSE,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  boot_basis = seq_along(Y),
  return_intermediaries = TRUE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
  ordinal = FALSE,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  min_split_correlation = 0,
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

`Y`	Numeric outcome vector.
`observables`	A list of matrices or data frames, one per latent predictor.
`covariates`	Optional matrix or data frame of observed covariates.
`observables_groupings`	Optional list of grouping vectors, one per latent predictor. Defaults to the column names of each observable matrix.
`make_observables_groupings`	Logical scalar or vector passed to `lpmec_onerun` for each latent predictor.
`n_boot`	Non-negative integer number of row-bootstrap iterations.
`n_partition`	Positive integer number of split-half partitions per original or bootstrap sample.
`partition_aggregation`	Aggregation strategy across partitions. See `lpmec`.

partition_aggregation_probs
: Quantile probabilities for winsorized or trimmed partition aggregation.

boot_basis
: Optional vector of indices or strata for row bootstrap.

return_intermediaries
: Logical. If TRUE, returns per-run coefficient matrices.

estimation_method
: Character scalar or vector passed to lpmec_onerun for each latent predictor.

latent_estimation_fn
: Optional function or list of functions used when estimation_method = "custom".

ordinal
: Logical scalar or vector passed to lpmec_onerun.

mcmc_control
: List passed to lpmec_onerun.

min_split_correlation
: Minimum allowed split-half correlation. The correction is defined only for positive componentwise correlations.

conda_env
: Character string naming the conda environment for MCMC methods.

conda_env_required
: Logical indicating whether conda_env is required.

Value

A list of class lpmec_multivariate containing aggregated uncorrected OLS and corrected IV latent coefficients with bootstrap uncertainty summaries when n_boot >= 1.

---

lpmec_multivariate_onerun

Multivariate latent-predictor measurement-error correction

Description

Implements the split-indicator multivariate IV correction for several latent predictors. Each latent predictor is estimated from its own indicator matrix.

Usage

```
lpmec_multivariate_onerun(
  Y,
  observables,
  covariates = NULL,
  observables_groupings = NULL,
  make_observables_groupings = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
  ordinal = FALSE,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  min_split_correlation = 0,
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

Y	Numeric outcome vector.
observables	A list of matrices or data frames, one per latent predictor.
covariates	Optional matrix or data frame of observed covariates.
observables_groupings	Optional list of grouping vectors, one per latent predictor. Defaults to the column names of each observable matrix.
make_observables_groupings	Logical scalar or vector passed to lpmec_onerun for each latent predictor.
estimation_method	Character scalar or vector passed to lpmec_onerun for each latent predictor.
latent_estimation_fn	Optional function or list of functions used when estimation_method = "custom".
ordinal	Logical scalar or vector passed to lpmec_onerun.
mcmc_control	List passed to lpmec_onerun.
min_split_correlation	Minimum allowed split-half correlation. The correction is defined only for positive componentwise correlations.
conda_env	Character string naming the conda environment for MCMC methods.
conda_env_required	Logical indicating whether conda_env is required.

Value

A list of class lpmec_multivariate_onerun containing uncorrected OLS coefficients, uncorrected IV coefficients, corrected IV coefficients, split-half correlations, first-stage diagnostics, and latent score matrices.

---

lpmec_onerun *lpmec_onerun*

Description

Implements analysis for latent variable models with measurement error correction

Usage

```
lpmec_onerun(
  Y,
  observables,
  observables_groupings = colnames(observables),
  make_observables_groupings = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
  batch_size = 512L, chain_method = "parallel", subsample_method = "full", n_thin_by =
  1L, n_chains = 2L, outcome_prior = list(calibration = "data"), joint2_prior = list()),
  ordinal = FALSE,
```

```
  conda_env = "lpmec",
  conda_env_required = FALSE,
  partition = NULL,
  partition_id = NULL
)
```

Arguments

`Y`	A vector of observed outcome variables
`observables`	A matrix of observable indicators used to estimate the latent variable
`observables_groupings`	A vector specifying groupings for the observable indicators. Default is column names of observables.
`make_observables_groupings`	Logical. If TRUE, creates dummy variables for each level of the observable indicators. Default is FALSE.
`estimation_method`	Character specifying the estimation approach. Options include: • "em" (default): Uses expectation-maximization via `emIRT` package. Supports both binary (via `emIRT::binIRT`) and ordinal (via `emIRT::ordIRT`) indicators. • "pca": First principal component of observables. • "averaging": Uses feature averaging. • "mcmc": Markov Chain Monte Carlo estimation using either `pscl::ideal` (R backend) or `numpyro` (Python backend) • "mcmc_joint": Joint Bayesian model that simultaneously estimates latent variables and outcome relationship using `numpyro` • "mcmc_joint2": NumPyro mixed factor-analysis benchmark with binary indicators and continuous `Y` in one factor model • "mcmc_overimputation": Two-stage MCMC approach with measurement error correction via over-imputation • "custom": In this case, latent estimation performed using `latent_estimation_fn`.
`latent_estimation_fn`	Custom function for estimating latent trait from `observables` if `estimation_method="custom"` (optional). The function should accept a matrix of observables (rows are observations) and return a numeric vector of length equal to the number of observations.
`mcmc_control`	A list indicating parameter specifications if MCMC used. `backend` Character string indicating the MCMC engine to use. Valid options are `"pscl"` (default, uses the R-based `pscl::ideal` function) or `"numpyro"` (uses the Python numpyro package via reticulate). `n_samples_warmup` Integer specifying the number of warm-up (burn-in) iterations before samples are collected. Default is `500`. `n_samples_mcmc` Integer specifying the number of post-warmup MCMC iterations to retain. Default is `1000`. `batch_size` Integer row subsample size used when `subsample_method = "batch"` with the NumPyro backend. Must be between `1` and `nrow(observables) - 1`. Default is `512`.

subsample_method Character string for NumPyro likelihood evaluation. Use "full" (default) for all rows or "batch" for experimental HMCECS row subsampling with batch_size.

chain_method Character string passed to numpyro specifying how to run multiple chains. Options: "parallel" (default), "sequential", or "vectorized".

anchor_parameter_id Optional 1-based observable-column index used by NumPyro MCMC backends to anchor item difficulty orientation. If omitted or NULL, automatic orientation is used.

n_thin_by Integer indicating the thinning factor for MCMC samples. Default is 1.

n_chains Integer specifying the number of parallel MCMC chains to run. Default is 2.

outcome_prior List controlling "mcmc_joint" outcome-model priors for the NumPyro backend. By default, calibration = "data" centers the intercept prior at mean(Y) and scales intercept, slope, and residual-sigma priors by sd(Y). Use calibration = "legacy" to restore the previous unit-scale priors, or provide numeric overrides for intercept_mean, intercept_sd, slope_mean, slope_sd, and sigma_sd. The optional scale_floor sets the minimum scale used for data-calibrated priors.

joint2_prior List controlling "mcmc_joint2" priors. Defaults are lambda_mean = 0, lambda_sd = 2, psi_shape = 0.0005, and psi_scale = 0.0005. For item loadings, lambda_mean and lambda_sd parameterize the raw loading scale before the positive softplus transform; all item loadings are positive by default.

ordinal Logical indicating whether the observable indicators are ordinal (TRUE) or binary (FALSE).

conda_env A character string specifying the name of the conda environment to use via reticulate. Default is "lpmec".

conda_env_required A logical indicating whether the specified conda environment must be strictly used. If TRUE, an error is thrown if the environment is not found. Default is FALSE.

partition Optional fixed split-half partition. When supplied, must be a list with split1_names and split2_names entries naming observable groups. Default is NULL, which preserves the historical one-off random split behavior.

partition_id Optional identifier for partition; stored in the returned object for bootstrap diagnostics.

Details

This function implements a latent variable analysis with measurement error correction. It splits the observable indicators into two sets, estimates latent variables using each set, and then applies various correction methods including OLS correction and instrumental variable approaches.

Value

A list containing various estimates and statistics:

- Naive, IV, corrected IV, and corrected OLS estimates: ols_*, iv_*, corrected_iv_*, and corrected_ols_*. Split-specific estimates use suffixes _a and _b.
- var_est_split: Estimated split-half measurement-error variance.

- bayesian_ols_*_outer_normed and bayesian_ols_*_inner_normed: MCMC coefficient summaries when an MCMC method is used; otherwise NA.
- m_stage_1_erv and m_reduced_erv: Extreme robustness values for the first-stage and reduced-form regressions.
- outcome_prior_*: Resolved outcome-prior values used by NumPyro joint outcome models.
- mcmc_joint2_*: NumPyro "mcmc_joint2" diagnostics, including effective-sample-size percentages, maximum R-hat, divergent transitions, mean accept probability, and orientation diagnostics.
- x_est1 and x_est2: Split-half latent variable estimates.

Standard Errors

The following single-run standard errors and t-statistics are currently returned as NA because their analytical derivation is not yet implemented:

- corrected_iv_se: Standard error for the corrected IV coefficient
- corrected_ols_se: Standard error for the corrected OLS coefficient
- corrected_ols_tstat: T-statistic for the corrected OLS coefficient
- corrected_ols_coef_alt: Alternative corrected OLS coefficient

For inference on these quantities, use the bootstrap approach via lpmec, which provides valid confidence intervals and standard errors through resampling.

Examples

```
# Generate some example data
set.seed(123)
Y <- rnorm(1000)
observables <- as.data.frame(matrix(sample(c(0,1), 1000*10, replace = TRUE), ncol = 10))

# Run the analysis
results <- lpmec_onerun(Y = Y,
                        observables = observables)

# View the corrected estimates
print(results)
```

---

lpmec_panel *Aggregated panel/fixed-effects measurement-error correction*

Description

Runs lpmec_panel_onerun on the original sample and on cluster (unit) bootstrap resamples, optionally over repeated split-half partitions for observables-mode measures, and aggregates the naive, corrected, and reliability quantities across runs. Units are resampled with replacement and each draw receives a fresh pseudo-id, so a unit drawn twice enters as two distinct clusters/fixed-effect groups; the whole pipeline (scoring, sign alignment, transforms, reliabilities, regressions, corrections) is re-run on every (bootstrap, partition) pair.

Usage

```
lpmec_panel(
  Y = NULL,
  unit = NULL,
  time = NULL,
  observables = NULL,
  scores = NULL,
  split_scores = NULL,
  covariates = NULL,
  Y_split_scores = NULL,
  design = c("within", "twoway", "difference", "pooled"),
  diff_k = 1L,
  estimation_method = "averaging",
  min_reliability = 0.05,
  min_cor_n = 30L,
  demean_iterations = 25L,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  return_intermediaries = TRUE,
  seed = NULL,
  ...
)
```

Arguments

`Y`	Numeric outcome vector (one value per panel row). Non-finite values are treated as missing. Optional when `Y_split_scores` is supplied (the outcome score is then built from the two halves).
`unit`	Vector of unit (cluster) identifiers, one per row. Required for the `"within"`, `"twoway"`, and `"difference"` designs (it defines the fixed-effect groups, the exact-gap differencing paths, and the cluster-robust standard errors). May be `NULL` for `design = "pooled"` (each row is then its own cluster, covering the pure cross-sectional case).
`time`	Optional numeric vector of time identifiers, one per row. Required for the `"twoway"` and `"difference"` designs. (unit, time) pairs must be unique.
`observables`	Optional list of item matrices or data frames, one per measure (a single matrix is treated as one measure). Each measure is scored via `lpmec_onerun` with `estimation_method` and must have at least 4 columns; measures with fewer components must be supplied through `split_scores`.
`scores`	Optional named list (or matrix/data frame with one column per measure) of pre-computed full measure scores.
`split_scores`	Optional named list of n x 2 matrices of half scores, one per measure.
`covariates`	Optional matrix or data frame of observed covariates; they are design-transformed together with the outcome and scores and enter both the OLS and both IV stages. Values must be finite.
`Y_split_scores`	Optional n x 2 matrix of outcome half scores for a *latent* outcome (Proposition 4). Each half is pooled z-scored; the outcome reliability `rho_Y` is the Spearman-Brown step-up of the half correlation, and every corrected estimator is additionally divided by `sqrt(rho_Y)`. When `Y` is not supplied, the outcome score is the

pooled z-score of the mean of the two z-scored halves; when `Y` is also supplied, it is used as the outcome score and the halves only estimate `rho_Y` (a sensitivity calculation unless the halves' error covariance matches that of `Y`).

`design` Panel design transform for estimation: one of `"within"` (unit demeaning), `"twoway"` (iterated unit-and-time demeaning), `"difference"` (exact-gap k-period differencing), or `"pooled"` (identity). Default `"within"`.

`diff_k` Positive integer gap for the `"difference"` design. Rows lacking a same-unit observation at exactly `time - diff_k` are set to `NA` rather than differenced against a nearer one.

`estimation_method` Estimation method(s) passed to `lpmec_onerun` for `observables`-mode measures (single value or one per measure). Default `"averaging"`.

`min_reliability` Reliability floor for the corrected-OLS division. Estimated reliabilities below this value (or negative) yield `NA` corrected OLS coefficients with a single warning. Default `0.05`.

`min_cor_n` Minimum number of complete pairs required to report any correlation. Default `30`.

`demean_iterations` Number of iterated demeaning passes for the `"twoway"` design (one pass is exact only on balanced panels). Default `25`.

`n_boot` Non-negative integer number of cluster-bootstrap replications. Default `32`.

`n_partition` Positive integer number of split-half partitions per original or bootstrap sample. Partitions only vary for `observables`-mode measures; without `observables` the value is coerced to 1 with a message. Default `10`.

`partition_aggregation` Aggregation strategy across partitions within each bootstrap replication: `"median"`, `"winsorized_mean"`, `"trimmed_mean"`, or a function. See `lpmec`.

`partition_aggregation_probs` Quantile probabilities for winsorized or trimmed partition aggregation.

`return_intermediaries` Logical. If `TRUE`, returns the per-run coefficient matrices as `Intermediary_*` entries along with `Intermediary_BootIndex` and `Intermediary_PartitionIndex`.

`seed` Optional seed applied locally to the scoring partitions and the bootstrap (the caller's random-number state is restored on exit).

`...` Additional arguments passed to `lpmec_onerun` for `observables`-mode measures (e.g., `ordinal`, `mcmc_control`).

Details

Aggregation follows the package convention: per-run values are first aggregated across partitions within each bootstrap replication (`partition_aggregation`), then the original-sample aggregate is the point estimate and the bootstrap aggregates provide the standard error and percentile interval. Bootstrap replications that fail are caught, recorded as `NA` rows, counted in `boot_n_failed`, and excluded from the bootstrap summaries.

See `lpmec_panel_onerun` for the correction algebra, including the "IV multiplies, OLS divides" asymmetry and the covariate caveat.

Value

A list of class lpmec_panel. For each aggregated quantity <field> of lpmec_panel_onerun – ols_coef, corrected_ols_coef(_split/_triad/_pair), corrected_ols_lower/upper, sd_design_x, ols_coef_local, corrected_ols_coef_local(_split/_triad/_pair), corrected_ols_local_lower/upper, iv_coef, corrected_iv_coef(_within/_cross), split_correlation, design_split_correlation, split_rho_score, design_split_rho_score, triad, design_triad, pair_correlation, design_pair_correlati rho_Y_half, rho_Y, first_stage_fstat, cross_iv_coef, corrected_cross_iv_coef, corrected_cross_iv_pair – the result holds the original-sample partition-aggregated estimate <field> plus bootstrap <field>_se, <field>_lower, and <field>_upper (percentile 2.5%/97.5%). Also included: ols_cluster_se (analytic cluster-robust SE from the original sample), the original-sample reliability table, cor_matrices, cor_n_matrices, x_est, x_est1, x_est2, corrected_ols_source, corrected_ols_local_source, corrected_iv_source, metadata (measure_names, n_measures, design, diff_k, n_obs, n_units, n_boot, n_partition, boot_n_failed), and – when return_intermediaries = TRUE – Intermediary_BootIndex, Intermediary_PartitionIndex, and one Intermediary_<field> matrix of per-run values (rows ordered as the runs; bootstrap runs that failed are NA rows).

Examples

```
set.seed(100)
n_units <- 60
n_periods <- 12
unit <- rep(seq_len(n_units), times = n_periods)
time <- rep(seq_len(n_periods), each = n_units)
latent <- rnorm(n_units)[unit] + rnorm(n_units * n_periods, sd = 0.7)
half1 <- latent + rnorm(n_units * n_periods, sd = 0.5)
half2 <- latent + rnorm(n_units * n_periods, sd = 0.5)
Y <- 0.4 * latent + rnorm(n_units)[unit] +
  rnorm(n_units * n_periods)

result <- lpmec_panel(
  Y = Y, unit = unit, time = time,
  split_scores = list(m1 = cbind(half1, half2)),
  design = "within",
  n_boot = 4L,
  seed = 1
)
result$corrected_ols_coef
result$corrected_ols_coef_se
```

---

lpmec_panel_onerun *Single-run panel/fixed-effects measurement-error correction*

Description

Estimates the effect of a latent predictor on a panel outcome under a fixed-effects or differencing design, and corrects the naive coefficient for the design-amplified attenuation caused by measurement error in the latent-variable scores (Propositions 1, 3, and 7 of the accompanying working paper; Proposition 2a for the design-local scale and Proposition 4 for latent outcomes). Latent measures may be supplied as raw item batteries (observables, scored via lpmec_onerun), as pre-computed half scores (split_scores), or as full pre-computed scores (scores); the three sources are merged by measure name.

Usage

```
lpmec_panel_onerun(
  Y = NULL,
  unit = NULL,
  time = NULL,
  observables = NULL,
  scores = NULL,
  split_scores = NULL,
  covariates = NULL,
  Y_split_scores = NULL,
  design = c("within", "twoway", "difference", "pooled"),
  diff_k = 1L,
  estimation_method = "averaging",
  min_reliability = 0.05,
  min_cor_n = 30L,
  demean_iterations = 25L,
  partition = NULL,
  ...
)
```

Arguments

`Y`	Numeric outcome vector (one value per panel row). Non-finite values are treated as missing. Optional when `Y_split_scores` is supplied (the outcome score is then built from the two halves).
`unit`	Vector of unit (cluster) identifiers, one per row. Required for the `"within"`, `"twoway"`, and `"difference"` designs (it defines the fixed-effect groups, the exact-gap differencing paths, and the cluster-robust standard errors). May be `NULL` for `design = "pooled"` (each row is then its own cluster, covering the pure cross-sectional case).
`time`	Optional numeric vector of time identifiers, one per row. Required for the `"twoway"` and `"difference"` designs. (unit, time) pairs must be unique.
`observables`	Optional list of item matrices or data frames, one per measure (a single matrix is treated as one measure). Each measure is scored via `lpmec_onerun` with `estimation_method` and must have at least 4 columns; measures with fewer components must be supplied through `split_scores`.
`scores`	Optional named list (or matrix/data frame with one column per measure) of pre-computed full measure scores.
`split_scores`	Optional named list of n x 2 matrices of half scores, one per measure.
`covariates`	Optional matrix or data frame of observed covariates; they are design-transformed together with the outcome and scores and enter both the OLS and both IV stages. Values must be finite.
`Y_split_scores`	Optional n x 2 matrix of outcome half scores for a *latent* outcome (Proposition 4). Each half is pooled z-scored; the outcome reliability `rho_Y` is the Spearman-Brown step-up of the half correlation, and every corrected estimator is additionally divided by `sqrt(rho_Y)`. When `Y` is not supplied, the outcome score is the pooled z-score of the mean of the two z-scored halves; when `Y` is also supplied, it is used as the outcome score and the halves only estimate `rho_Y` (a sensitivity calculation unless the halves' error covariance matches that of `Y`).

`design` Panel design transform for estimation: one of `"within"` (unit demeaning), `"twoway"` (iterated unit-and-time demeaning), `"difference"` (exact-gap k-period differencing), or `"pooled"` (identity). Default `"within"`.

`diff_k` Positive integer gap for the `"difference"` design. Rows lacking a same-unit observation at exactly `time - diff_k` are set to `NA` rather than differenced against a nearer one.

`estimation_method` Estimation method(s) passed to `lpmec_onerun` for `observables`-mode measures (single value or one per measure). Default `"averaging"`.

`min_reliability` Reliability floor for the corrected-OLS division. Estimated reliabilities below this value (or negative) yield `NA` corrected OLS coefficients with a single warning. Default `0.05`.

`min_cor_n` Minimum number of complete pairs required to report any correlation. Default `30`.

`demean_iterations` Number of iterated demeaning passes for the `"twoway"` design (one pass is exact only on balanced panels). Default `25`.

`partition` Optional named list of fixed split-half partitions, keyed by measure name, passed to `lpmec_onerun` for `observables`-mode measures.

`...` Additional arguments passed to `lpmec_onerun` for `observables`-mode measures (e.g., `ordinal`, `mcmc_control`).

Details

Under the panel design transform, classical measurement error attenuates the naive coefficient by the design reliability `rho_design` of the (pooled-standardized) score, while the pooled standardization inflates it by `1 / sqrt(rho_pooled)`, so `plim b = beta * sqrt(rho_pooled) * rho_design / rho_pooled` simplifies to `beta * rho_design / sqrt(rho_pooled)` and the corrected OLS coefficient is `b * sqrt(rho_pooled) / rho_design`. Fixed-effects and differencing transforms strip the persistent part of the latent signal but not the transient measurement error, so `rho_design` is typically far below `rho_pooled` and pooled corrections badly understate the bias.

Score-scale matching (Assumption 3). The reliabilities entering the OLS division must be those of the score actually regressed. The raw half-split correlation estimates the reliability of a *half* score, while the regression uses the full score (the average of the two z-scored halves, or the full-battery estimate), so the split-based correction uses the Spearman-Brown step-up `2r / (1 + r)` of both the pooled and the design split correlations. The step-up is exact when the full score is the average of two independent parallel halves and a parallel-forms approximation for full-battery estimates. When a measure merges a user-supplied full score with separate half scores (`scores + split_scores`), the stepped-up value matches the supplied score's error covariance only under that same parallel-forms assumption, so the split-based correction is then a sensitivity calculation.

IV multiplies, OLS divides. The corrected OLS coefficient divides by the design reliability, which is estimated with noise and can be near zero under aggressive transforms; corrections with `rho_design < min_reliability` (or with the pooled reliability below the floor) are therefore reported as `NA` rather than divided out. The split-IV correction instead multiplies the design-transformed IV coefficient by `sqrt(rho_pooled)`: the instrument (the other half score or another measure) removes the design-reliability denominator, so the corrected IV estimate stays stable exactly where the OLS correction becomes fragile. Do not swap the two operations.

Reliability variants for the OLS correction: the Spearman-Brown stepped-up within-measure split reliability (exceeds construct-relevant reliability when the halves share systematic error, Proposition

7a), the cross-measure triad `r_ml * r_mk / r_lk` (requires 3+ measures; consistent under pairwise-orthogonal total errors, Proposition 7b, and a lower bound only under the target-specific ratio condition of Proposition 7c – shared error can move it in either direction), and – only for a two-measure system where a measure has no split halves – the cross-measure pair correlation, which identifies the common reliability under a parallel-measures assumption. `corrected_ols_lower` and `corrected_ols_upper` span the finite variants; this is a sensitivity range, not a partial-identification interval, unless the corresponding Proposition 7 condition is maintained.

Covariate caveat. With covariates, the scalar OLS correction is exact only when the covariates are uncorrelated with the latent predictor; otherwise the measurement error also contaminates the covariate coefficients and the correction of the latent slope is approximate. The split-IV estimator remains consistent for the latent slope in the presence of exogenous covariates, which enter both stages.

Value

A list of class `lpmec_panel_onerun` containing, with one named entry per measure unless noted:

- `ols_coef`, `ols_se`, `ols_n_obs`, `ols_n_clusters`: naive design-transformed OLS coefficient of `Y` on the measure score with cluster-robust (by unit) standard error.
- `corrected_ols_coef`, `corrected_ols_coef_split`, `corrected_ols_coef_triad`, `corrected_ols_coef_pair`, `corrected_ols_lower`, `corrected_ols_upper`, `corrected_ols_source`: corrected OLS coefficient(s) `b * sqrt(rho_pooled) / rho_design` per reliability variant (the split variant uses the Spearman-Brown stepped-up reliabilities `split_rho_score`/`design_split_rho_score`), the headline value (split when available, else triad, else the two-measure parallel pair), and the [min, max] sensitivity range over the finite variants.
- `sd_design_x`, `ols_coef_local`, `corrected_ols_coef_local(_split/_triad/_pair)`, `corrected_ols_local`, `corrected_ols_local_source`: design-local-scale quantities (Proposition 2a): the sd of the design-transformed score on the estimation sample, the naive local slope `ols_coef * sd_design_x` (effect per SD of the *transformed* latent trait), and its corrections `ols_coef_local / sqrt(rho_design)` per design-scale reliability variant. The local estimand moves with the design and is not comparable across specifications.
- `split_correlation`, `design_split_correlation`, `split_rho_score`, `design_split_rho_score`, `triad`, `design_triad`, `pair_correlation`, `design_pair_correlation`: reliability estimates under the pooled and estimation designs; `*_rho_score` are the Spearman-Brown step-ups of the raw half-split correlations to the full-score scale.
- `rho_Y_half`, `rho_Y`: with `Y_split_scores`, the raw outcome half-score correlation and its Spearman-Brown step-up (`NA` for an observed outcome).
- `iv_coef_a`, `iv_coef_b`, `iv_coef`, `iv_se_a`, `iv_se_b`, `first_stage_fstat_a`, `first_stage_fstat_b`: within-measure split-IV fits (half 1 instrumented by half 2 and the reverse) on the design-transformed data, with clustered first-stage F statistics.
- `corrected_iv_coef_a`, `corrected_iv_coef_b`, `corrected_iv_coef_within`: within-measure IV corrected by `sqrt` of the pooled split correlation.
- `cross_iv_coef`, `cross_iv_se`, `cross_first_stage_fstat`, `corrected_cross_iv_coef` (named `"<regressor>_by_<instrument>"`), `corrected_cross_iv_pair` (direction-averaged, named `"<a>_x_<b>"`), `corrected_iv_coef_cross` (per regressor, averaged over instruments): cross-measure IV fits with the target-oriented correction of Proposition 3c – `sqrt` of the regressor (target) measure's pooled triad reliability when 3+ measures are supplied, or `sqrt` of the pooled pair correlation under the two-measure parallel-measures fallback.
- `corrected_iv_coef`, `corrected_iv_source`, `first_stage_fstat`: headline corrected IV per measure (within-measure when available, else cross-measure) and the minimum first-stage F statistic among the directions it averages.

- `reliability`: data frame with one row per (design, measure) holding `split_correlation`, `split_n`, `rho_split`, `triad`, `rho_lo`, `rho_hi` (the Proposition-7 sensitivity range).
- `cor_matrices`, `cor_n_matrices`: per-design cross-measure correlation matrices and complete-pair counts.
- `x_est`, `x_est1`, `x_est2`: pooled z-scored, sign-aligned measure scores and half scores; `scalar_runs`: the underlying `lpmec_onerun` fits for `observables`-mode measures.
- Metadata: `measure_names`, `n_measures`, `measure_source`, `has_splits`, `sign_flipped`, `covariate_names`, `design`, `diff_k`, `n_obs`, `n_units`, `min_reliability`, `min_cor_n`, `demean_iterations`.

Examples

```
set.seed(100)
n_units <- 60
n_periods <- 12
unit <- rep(seq_len(n_units), times = n_periods)
time <- rep(seq_len(n_periods), each = n_units)
latent <- rnorm(n_units)[unit] + rnorm(n_units * n_periods, sd = 0.7)
half1 <- latent + rnorm(n_units * n_periods, sd = 0.5)
half2 <- latent + rnorm(n_units * n_periods, sd = 0.5)
Y <- 0.4 * latent + rnorm(n_units)[unit] +
  rnorm(n_units * n_periods)

run <- lpmec_panel_onerun(
  Y = Y, unit = unit, time = time,
  split_scores = list(m1 = cbind(half1, half2)),
  design = "within"
)
run$ols_coef
run$corrected_ols_coef
run$corrected_iv_coef
```

---

lpmec_reliability_bounds

Reliability sensitivity diagnostics for latent measures under panel design transforms

---

Description

Estimates, for each latent measure and each requested design transform, the within-measure split correlation (with its Spearman-Brown step-up to the full-score scale, `rho_split`) and the cross-measure triad reliability, and reports the sensitivity range `[rho_lo, rho_hi]` spanned by the two candidates for the measure's design reliability (Proposition 7 of the accompanying working paper). The range is a sensitivity diagnostic, not a partial-identification interval, unless Proposition 7's corresponding orthogonality or ratio condition is maintained. Optionally attaches a cluster (unit) bootstrap for all reported reliability quantities.

Usage

```
lpmec_reliability_bounds(
  observables = NULL,
  scores = NULL,
  split_scores = NULL,
  unit = NULL,
  time = NULL,
  designs = "pooled",
  diff_k = 1L,
  estimation_method = "averaging",
  min_reliability = 0.05,
  min_cor_n = 30L,
  demean_iterations = 25L,
  n_boot = 0L,
  seed = NULL,
  ...
)
```

Arguments

observables	Optional list of item matrices or data frames, one per measure (a single matrix is treated as one measure). Each measure is scored via lpmec_onerun with estimation_method, and its full-battery and split-half latent scores are harvested. Each item matrix must have at least 4 columns; measures with fewer components must be supplied through split_scores.
scores	Optional named list (or matrix/data frame with one column per measure) of precomputed full measure scores. Scores are pooled z-scored; split-based quantities are unavailable for these measures.
split_scores	Optional named list of n x 2 matrices of half scores, one per measure. Each half is pooled z-scored and the full-measure score is the z-score of the mean of the two z-scored halves. This is the required input mode for measures with only 2 components.
unit	Optional vector of unit (cluster) identifiers, one per row. Required for the "within", "twoway", and "difference" designs. When NULL, each row is its own cluster and only "pooled" designs are allowed.
time	Optional numeric vector of time identifiers, one per row. Required for the "twoway" and "difference" designs. (unit, time) pairs must be unique.
designs	Character vector of design transforms; any of "pooled" (identity), "within" (unit demeaning), "twoway" (iterated unit-and-time demeaning), and "difference" (exact-gap k-period differencing). Default "pooled".
diff_k	Positive integer gap for the "difference" design. Rows lacking a same-unit observation at exactly time - diff_k are set to NA rather than differenced against a nearer observation.
estimation_method	Estimation method(s) passed to lpmec_onerun for observables-mode measures (single value or one per measure). Default "averaging".
min_reliability	Reliability floor. Estimated reliabilities below this value (or negative) are excluded from the bounds and reported as NA with a single warning, since corrections that divide by such values are not identified in practice. Default 0.05.

`min_cor_n` Minimum number of complete pairs required to report any correlation; correlations on fewer pairs are `NA`. Default `30`.

`demean_iterations` Number of iterated demeaning passes for the `"twoway"` design (one pass is exact only on balanced panels). Default `25`.

`n_boot` Non-negative integer number of cluster-bootstrap replications. Units are resampled with replacement and receive fresh pseudo-ids (a unit drawn twice enters as two distinct clusters); the whole pipeline (scoring, sign alignment, transforms, reliabilities, bounds) is re-run on each draw. Default `0`.

`seed` Optional seed applied locally to the scoring partitions and the bootstrap (the caller's random-number state is restored on exit).

`...` Additional arguments passed to `lpmec_onerun` for `observables`-mode measures (e.g., `ordinal`, `mcmc_control`).

Details

The split correlation between two half scores of the same measure consistently estimates the reliability of a *half* score; `rho_split = 2r/(1+r)` steps it up to the scale of the full score (exact when the full score is the average of two independent parallel halves, a parallel-forms approximation otherwise). The two candidates entering the range therefore both target the full score's reliability. Their directional interpretation is model-dependent (Proposition 7): when the halves share a common systematic error component orthogonal to the trait, the split-based value exceeds construct-relevant reliability (Prop 7a); the triad `rho*_m = r_ml * r_mk / r_lk` equals it exactly when the measures' total errors are pairwise orthogonal (Prop 7b) and is a lower bound only under the target-specific ratio condition of Prop 7c – shared error among the other two measures pushes it down, but shared error involving the target measure can push it up. Absent those conditions, `[rho_lo, rho_hi]` is the [min, max] of two sensitivity candidates, not an identified set. With fewer than 3 measures the triad is not available and both endpoints collapse to `rho_split`. Correlations are computed after pooled z-scoring and sign alignment of all measure scores; correlations with fewer than `min_cor_n` complete pairs, and reliabilities below `min_reliability`, are reported as `NA` rather than propagated into unstable divisions.

Value

A list of class `lpmec_reliability_bounds` containing:

- `reliability`: data frame with one row per (design, measure) holding `split_correlation`, `split_n`, `rho_split` (the Spearman-Brown step-up of the split correlation to the full-score scale), `triad`, `rho_lo`, `rho_hi`, and – when `n_boot > 0` – bootstrap `_se`, `_lower`, and `_upper` columns for each of these reliability quantities.
- `cor_matrices` and `cor_n_matrices`: per-design cross-measure correlation matrices and complete-pair counts.
- `x_est`, `x_est1`, `x_est2`: pooled z-scored, sign-aligned measure score matrices (halves are `NA` for `scores`-mode measures).
- `measure_names`, `n_measures`, `designs`, `measure_source`, `sign_flipped`, `n_obs`, `n_units`, `diff_k`, `demean_iterations`, `min_reliability`, `min_cor_n`, `n_boot`, `n_boot_failed`.
- When `n_boot > 0`: `Intermediary_BootIndex` and per-replication matrices `Intermediary_split_correlation`, `Intermediary_rho_split`, `Intermediary_triad`, `Intermediary_rho_lo`, `Intermediary_rho_hi` (row 1 is the original sample).

Examples

```
set.seed(100)
n <- 400
latent <- rnorm(n)
half1 <- latent + rnorm(n, sd = 0.6)
half2 <- latent + rnorm(n, sd = 0.6)
score2 <- latent + rnorm(n, sd = 0.7)
score3 <- latent + rnorm(n, sd = 0.7)

bounds <- lpmec_reliability_bounds(
  split_scores = list(measure1 = cbind(half1, half2)),
  scores = list(measure2 = score2, measure3 = score3)
)
bounds$reliability
```

---

plot.lpmec *Plot method for lpmec objects*

Description

Creates visualizations of LPMEC model results. Can plot either the latent variable estimates or the bootstrap distribution of coefficients.

Usage

```
## S3 method for class 'lpmec'
plot(x, type = "latent", ...)
```

Arguments

x	An object of class lpmec returned by lpmec.
type	Character string specifying the plot type. Either "latent" (default) for a scatter plot of split-half latent estimates, or "coefficients" for a density plot of bootstrap coefficient estimates.
...	Additional arguments passed to plot or density.

Value

No return value, called for side effects (creates a plot).

See Also

lpmec, summary.lpmec, print.lpmec

plot.lpmec_moderator *Plot method for lpmec_moderator objects*

Description

Creates a scatter plot of the two pooled half scores of the first measure with split halves; when no measure has half scores, falls back to a scatter plot of the first two measure scores.

Usage

```
## S3 method for class 'lpmec_moderator'
plot(x, ...)
```

Arguments

x	An object of class lpmec_moderator.
...	Additional arguments passed to plot.

Value

No return value, called for side effects (creates a plot).

plot.lpmec_onerun *Plot method for lpmec_onerun objects*

Description

Creates a scatter plot comparing the two split-half latent variable estimates.

Usage

```
## S3 method for class 'lpmec_onerun'
plot(x, ...)
```

Arguments

x	An object of class lpmec_onerun returned by lpmec_onerun.
...	Additional arguments passed to plot.

Value

No return value, called for side effects (creates a plot).

See Also

lpmec_onerun, summary.lpmec_onerun, print.lpmec_onerun

plot.lpmec_panel *Plot method for lpmec_panel objects*

Description

Creates a scatter plot of the two pooled half scores of the first measure with split halves; when no measure has half scores, falls back to a scatter plot of the first two measure scores.

Usage

```
## S3 method for class 'lpmec_panel'
plot(x, ...)
```

Arguments

x	An object of class lpmec_panel.
...	Additional arguments passed to plot.

Value

No return value, called for side effects (creates a plot).

print.lpmec *Print method for lpmec objects*

Description

Prints a concise summary of bootstrapped LPMEC model results.

Usage

```
## S3 method for class 'lpmec'
print(x, ...)
```

Arguments

x	An object of class lpmec returned by lpmec.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

See Also

lpmec, summary.lpmec, plot.lpmec

print.lpmec_moderator *Print method for lpmec_moderator objects*

Description

Print method for lpmec_moderator objects

Usage

```
## S3 method for class 'lpmec_moderator'
print(x, ...)
```

Arguments

x	An object of class lpmec_moderator.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_moderator_onerun *Print method for lpmec_moderator_onerun objects*

Description

Print method for lpmec_moderator_onerun objects

Usage

```
## S3 method for class 'lpmec_moderator_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_moderator_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_multivariate *Print method for lpmec_multivariate objects*

Description

Print method for lpmec_multivariate objects

Usage

```
## S3 method for class 'lpmec_multivariate'
print(x, ...)
```

Arguments

x	An object of class lpmec_multivariate.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_multivariate_onerun *Print method for lpmec_multivariate_onerun objects*

Description

Print method for lpmec_multivariate_onerun objects

Usage

```
## S3 method for class 'lpmec_multivariate_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_multivariate_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_onerun *Print method for lpmec_onerun objects*

Description

Prints a concise summary of single-run LPMEC model results.

Usage

```
## S3 method for class 'lpmec_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_onerun returned by lpmec_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

See Also

lpmec_onerun, summary.lpmec_onerun, plot.lpmec_onerun

print.lpmec_panel *Print method for lpmec_panel objects*

Description

Print method for lpmec_panel objects

Usage

```
## S3 method for class 'lpmec_panel'
print(x, ...)
```

Arguments

x	An object of class lpmec_panel.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_panel_onerun *Print method for lpmec_panel_onerun objects*

Description

Print method for lpmec_panel_onerun objects

Usage

```
## S3 method for class 'lpmec_panel_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_panel_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_reliability_bounds *Print method for lpmec_reliability_bounds objects*

Description

Print method for lpmec_reliability_bounds objects

Usage

```
## S3 method for class 'lpmec_reliability_bounds'
print(x, ...)
```

Arguments

x	An object of class lpmec_reliability_bounds.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

summary.lpmec *Summary method for lpmec objects*

Description

Provides a comprehensive summary of bootstrapped LPMEC model results including OLS, IV, corrected, and Bayesian coefficient estimates with confidence intervals.

Usage

```
## S3 method for class 'lpmec'
summary(object, ...)
```

Arguments

object	An object of class lpmec returned by lpmec.
...	Additional arguments (currently unused).

Value

A data frame containing coefficient estimates, standard errors, and confidence intervals, returned invisibly. The data frame has rows for OLS, IV, Corrected IV, Corrected OLS, and Bayesian OLS estimates.

See Also

lpmec, print.lpmec, plot.lpmec

summary.lpmec_moderator *Summary method for lpmec_moderator objects*

Description

Summary method for lpmec_moderator objects

Usage

```
## S3 method for class 'lpmec_moderator'
summary(object, ...)
```

Arguments

object	An object of class lpmec_moderator.
...	Additional arguments (currently unused).

Value

A data frame of naive and corrected interaction-regression coefficient estimates and reliability quantities with bootstrap standard errors and percentile confidence intervals, returned invisibly.

summary.lpmec_moderator_onerun *Summary method for lpmec_moderator_onerun objects*

Description

Summary method for lpmec_moderator_onerun objects

Usage

```
## S3 method for class 'lpmec_moderator_onerun'
summary(object, ...)
```

Arguments

object	An object of class lpmec_moderator_onerun.
...	Additional arguments (currently unused).

Value

A data frame of naive and corrected interaction-regression coefficient estimates, returned invisibly. When three or more measures are supplied, the per-measure triad corrections are also printed.

summary.lpmec_multivariate *Summary method for lpmec_multivariate objects*

Description

Summary method for lpmec_multivariate objects

Usage

```
## S3 method for class 'lpmec_multivariate'
summary(object, ...)
```

Arguments

object	An object of class lpmec_multivariate.
...	Additional arguments (currently unused).

Value

A data frame of aggregated latent-predictor coefficient estimates.

summary.lpmec_multivariate_onerun *Summary method for lpmec_multivariate_onerun objects*

Description

Summary method for lpmec_multivariate_onerun objects

Usage

```
## S3 method for class 'lpmec_multivariate_onerun'
summary(object, ...)
```

Arguments

object	An object of class lpmec_multivariate_onerun.
...	Additional arguments (currently unused).

Value

A data frame of latent-predictor coefficient estimates.

summary.lpmec_onerun *Summary method for lpmec_onerun objects*

Description

Provides a summary of single-run LPMEC model results including OLS, IV, and corrected coefficient estimates.

Usage

```
## S3 method for class 'lpmec_onerun'
summary(object, ...)
```

Arguments

object	An object of class lpmec_onerun returned by lpmec_onerun.
...	Additional arguments (currently unused).

Value

A data frame containing coefficient estimates and standard errors, returned invisibly. The data frame has rows for OLS, IV, Corrected IV, and Corrected OLS estimates.

See Also

lpmec_onerun, print.lpmec_onerun, plot.lpmec_onerun

summary.lpmec_panel *Summary method for lpmec_panel objects*

Description

Summary method for lpmec_panel objects

Usage

```
## S3 method for class 'lpmec_panel'
summary(object, ...)
```

Arguments

object	An object of class lpmec_panel.
...	Additional arguments (currently unused).

Value

A data frame with one row per measure holding the naive OLS coefficient, the headline corrected OLS coefficient with its [min, max] reliability-variant interval, the corrected IV coefficient (with cluster-bootstrap standard errors), the pooled split and triad reliabilities, and the first-stage F statistic, returned invisibly.

summary.lpmec_panel_onerun *Summary method for lpmec_panel_onerun objects*

Description

Summary method for lpmec_panel_onerun objects

Usage

```
## S3 method for class 'lpmec_panel_onerun'
summary(object, ...)
```

Arguments

object	An object of class lpmec_panel_onerun.
...	Additional arguments (currently unused).

Value

A data frame with one row per measure holding the naive OLS coefficient (with cluster-robust standard error), the headline corrected OLS coefficient with its [min, max] reliability-variant interval, the corrected IV coefficient, the pooled split and triad reliabilities, and the first-stage F statistic, returned invisibly.

Index